TP4 — Composant adaptatif : container queries

Module CSS avancé · Licence 3 · Jour 2 · à partir de 11h30 (après le Cours 4)

Objectifs pédagogiques

À l'issue de ce TP, vous devez être capable de :

1. **expliquer pourquoi une media query ne suffit pas** pour un composant réutilisable ;
2. **déclarer un conteneur** (`container-type: inline-size`, `container-name`) et écrire des requêtes `@container` ;
3. **dimensionner relativement au conteneur** avec l'unité `cqi`, et rendre une typographie fluide avec `clamp()` ;
4. **utiliser l'unité `dvh`** et les **propriétés logiques** (`margin-inline`, `padding-block`, `inset-inline-start`…), et vérifier le résultat en **RTL** ;
5. **appliquer les bases de l'accessibilité** : `:focus-visible`, cibles ≥ 44 px, contrastes AA, `prefers-reduced-motion`.

Prérequis

- Cours 4.
- Grid et `grid-template-areas` (TP2).
- DevTools : mode responsive, sélecteur de couleur, onglet *Rendering*.

Durée et organisation

Durée	1 h (5 min de lecture + 6 étapes ; exercice d'approfondissement facultatif)
Organisation	en groupe de trois (trinôme). Alternez au clavier à chaque étape ; chaque membre doit coder.
Fichiers	`starter/index.html` (ne pas modifier, sauf `dir="rtl"` à l'étape 5) · `starter/style.css` (à compléter)
Navigateur	Chrome / Edge récent (Firefox pour comparer) — double-clic sur `index.html`, pas de serveur

Livrable et critères de réussite

- **Livrable** : le fichier `starter/style.css` complété, présenté à l'enseignant en fin de séance (page ouverte à 1280 px puis à 375 px).
- **Critères de réussite** : la liste détaillée figure en fin d'énoncé. Servez-vous-en pour vous auto-évaluer au fil des étapes ; l'enseignant la vérifie en une minute. Ce TP relève de l'évaluation formative : il n'est pas noté.
- L'exercice d'approfondissement est **facultatif**.

Contexte de l'exercice

La page programme de *La Nuit des Jeux Indés* utilise **un seul composant**, la carte événement, dans trois contextes : un **bandeau « À la une »** très large, une **grille** de largeur moyenne et une **barre latérale** étroite. Réglée par `@media (min-width: 700px)`, la carte passe en horizontal dans tous les contextes dès que la fenêtre est large, y compris là où elle est trop étroite pour cela. Vous devez faire en sorte que la carte choisisse sa mise en page selon **la place dont elle dispose**, et non selon la taille de l'écran, **sans aucune media query de largeur**.

Étape 1 — Diagnostic : pourquoi la media query échoue (≈ 5 min)

1. Ouvrez `starter/index.html` en 1280 px, puis en 375 px (DevTools → mode responsive).
2. Observez les trois contextes : à 1280 px, **toutes** les cartes passent en horizontal, y compris celles de la grille, qui sont trop étroites pour ça.
3. Répondez en une phrase (dans un commentaire CSS) : *« À quoi réagit une media query ? À quoi devrait réagir la carte ? »*
4. Supprimez (ou commentez) tout le bloc `MISE EN PAGE DE LA CARTE — VERSION « MEDIA QUERY »`.

Indice

À 1280 px, la carte de la barre latérale ne mesure qu'environ 270 px et celle du bandeau près de 800 px, **dans la même fenêtre**. Une media query ne connaît que la fenêtre : elle ne peut pas les distinguer.

Étape 2 — Déclarer les conteneurs (≈ 5 min)

Faites de chaque `.event-card` un **conteneur de requête** nommé `carte`, qui n'observe que sa **largeur**. Ensuite, c'est son enfant `.event-card__inner` qui sera mis en page.

Indice 1 — la syntaxe

```
.event-card {
  container-type: inline-size;
  container-name: carte;
  /* ou le raccourci : container: carte / inline-size; */
}
```

Indice 2 — pourquoi pas sur .event-card__inner ? Pourquoi pas « size » ?

- Un élément **ne peut pas s'interroger lui-même** : les règles `@container` s'appliquent aux **descendants** du conteneur.
- `container-type: size` observe aussi la hauteur, mais impose que la hauteur ne dépende plus du contenu : sans hauteur fixée, la carte s'effondre à 0 px. `inline-size` suffit presque toujours.

Étape 3 — Trois mises en page avec `@container` (≈ 15 min)

On travaille en **mobile-first à l'échelle du composant** : le style par défaut est la version la plus étroite.

Largeur de la carte	Mise en page	Où la voit-on à 1280 px ?
< 18rem (par défaut)	**Compacte** : pas de visuel ni de description, heure en pastille au-dessus du titre, boutons pleine largeur	barre latérale
≥ 18rem	**Moyenne** : verticale, visuel 16/9, heure posée sur le visuel, description visible	grille
≥ 42rem	**Large** : horizontale, visuel à gauche sur toute la hauteur	bandeau « À la une »

Contrôle : à 1280 px, les 3 contextes affichent 3 mises en page différentes. Passez à 375 px : la barre latérale passe **sous** le programme, devient large… et ses cartes changent toutes seules de mise en page. Aucune ligne de CSS ne parle de la taille de l'écran.

Indice 1 — les grid-template-areas

```
/* A. compacte (par défaut) */
.event-card__inner {
  grid-template-columns: minmax(0, 1fr);
  grid-template-areas: "time" "body" "actions";
}
.event-card__visual, .event-card__desc { display: none; }

/* C. large */
@container carte (width >= 42rem) {
  .event-card__inner {
    grid-template-columns: minmax(14rem, 2fr) 3fr;
    grid-template-rows: 1fr auto;
    grid-template-areas:
      "visual body"
      "visual actions";
  }
}
```

Indice 2 — et l'heure posée sur le visuel ?

Vous avez supprimé le bloc qui la positionnait : reprenez-le dans la requête « moyenne ».

```
@container carte (width >= 18rem) {
  .event-card__time { position: absolute; top: 1.85rem; left: 1.85rem; } /* à rendre « logique » à l'étape 5 */
}
```

Indice 3 — l'ordre des requêtes compte

Avec `>=`, écrivez la requête 18rem **avant** la requête 42rem : à 50rem, les deux s'appliquent, et c'est la dernière qui l'emporte (cascade normale, même spécificité).

Indice 4 — ma requête ne s'applique pas

Vérifiez que l'élément ciblé est bien **à l'intérieur** du conteneur, et que le nom est identique (`@container carte`). Les DevTools Chrome affichent un badge **container** à côté de l'élément : cliquez dessus pour voir sa taille.

Étape 4 — Tailles fluides : `cqi`, `clamp()` et `dvh` (≈ 10 min)

1. **Titre de la carte** : `font-size: clamp(1rem, 0.8rem + 2.5cqi, 1.9rem);` (1 `cqi` = 1 % de la largeur du conteneur).
2. **Rembourrage de la carte** : `padding: clamp(0.75rem, 4cqi, 1.75rem);` ; et l'icône du visuel : `inline-size: clamp(2.5rem, 18cqi, 6rem)`.
3. **Titre du héros** (`h1`) : typo fluide liée à l'**écran** cette fois, avec `clamp()` et `vw`, entre 2.25rem et 5rem. Testez à 375 px.
4. **Hauteur du héros** : remplacez `min-height: 60vh` par `min-block-size: 60dvh`.

Contrôle : à 1280 px, le titre de la carte du bandeau est nettement plus grand que celui de la barre latérale, **sur le même écran**.

Indice — vw ou cqi ? vh, svh ou dvh ?

- `vw` = largeur de la **fenêtre** → pour la mise en page globale (héros). `cqi` = largeur du **conteneur** → pour l'intérieur d'un composant.
- Sur mobile, `100vh` ignore la barre d'adresse : le bas du héros est caché. `dvh` suit la hauteur **réellement visible** (elle change quand la barre se replie) ; `svh` prend la plus petite, stable, sans « saut ».
- Gardez toujours une part en `rem` dans `clamp()` : le zoom du navigateur doit continuer à agrandir le texte.

Étape 5 — Propriétés logiques et test RTL (≈ 10 min)

1. Repérez les `/* TODO étape 5 */` : `border-left`, `padding-left`, `margin-right`, `top`, `left`, `width`, `height`, `text-align: left`.
2. Remplacez-les par leurs équivalents **logiques** : `border-inline-start`, `padding-inline-start`, `margin-inline-end`, `inset-block-start`, `inset-inline-start`, `inline-size`, `block-size`, `text-align: start`.
3. **Test RTL** : DevTools → onglet *Elements* → double-cliquez sur `<html lang="fr">` et ajoutez `dir="rtl"`.

Contrôle : en RTL, **tout bascule en miroir proprement** : liseré coloré à droite, visuel du bandeau à droite, heure dans le coin supérieur droit du visuel, barre latérale à gauche avec sa bordure du bon côté. Refaites le test **avant** l'étape 5 (avec les propriétés physiques) : l'heure reste à gauche et tombe sur le texte.

Indice — pourquoi le visuel bascule-t-il tout seul ?

Grid, Flexbox et `grid-template-areas` suivent déjà le **sens d'écriture** : en RTL, la première colonne est à droite. Seules les propriétés **physiques** (`left` , `margin-right` ...) restent collées à leur côté. Les propriétés logiques parlent de « début / fin de ligne » (`inline`) et de « début / fin de bloc » (`block`).

La ponctuation française apparaît bizarrement en RTL (point en début de ligne) : c'est normal, le texte n'est pas en arabe ou en hébreu. Seule la mise en page nous intéresse ici.

Étape 6 — Accessibilité (≈ 10 min)

1. **Focus** : supprimez `a:focus { outline: none; }` et écrivez une règle `:focus-visible` (anneau de 3 px, `outline-offset`). Testez au clavier avec Tab.
2. **Cibles tactiles** : les boutons font environ 26 px de haut. Donnez-leur `min-block-size: 2.75rem` (44 px) et `min-inline-size: 2.75rem` .
3. **Contrastes AA** : avec le sélecteur de couleur des DevTools (ratio affiché), corrigez `--c-text-muted` (2,5:1) et `--c-warning` (2,1:1) pour atteindre **≥ 4,5:1** sur blanc.
4. **Mouvement** : sous `@media (prefers-reduced-motion: reduce)` , coupez le soulèvement des cartes au survol et l'animation du visuel « À la une ». Testez via DevTools → *Rendering* → *Emulate CSS media feature prefers-reduced-motion*.

Indice — des valeurs qui passent

En `oklch` , une clarté autour de 48-52 % sur fond blanc donne environ 5 à 7:1. Par exemple `oklch(48% 0.02 280)` pour le texte secondaire et `oklch(52% 0.14 50)` pour l'orange.

Indice — « mais c'est une media query ! »

`prefers-reduced-motion` interroge une **préférence de l'utilisateur**, pas la largeur de l'écran. La contrainte « zéro media query » vise les media queries de **largeur**.

Exercice d'approfondissement (facultatif) — Le conteneur nommé « sidebar » : mode frise

Quand la carte se trouve dans la **barre latérale** et que **cette barre** est étroite (< 18rem), la liste doit devenir une **frise chronologique** : un trait pointillé vertical côté début de ligne, une pastille colorée devant chaque carte, et le bouton « Détails » masqué. Sur mobile (barre latérale large, sous le contenu), la frise disparaît.

Contraintes : sans ajouter de classe dans le HTML, uniquement avec `@container` et des propriétés logiques (la frise doit passer à droite en RTL).

Indices

- Faites de `.sidebar` un **deuxième** conteneur : `container: sidebar / inline-size;`.
- Une requête **sans nom** vise le conteneur **le plus proche** de l'élément… c'est-à-dire la carte elle-même ! Le nom permet de viser un ancêtre précis : `@container sidebar (width < 18rem) { … }`.
- Pastille : `li::before` en `position: absolute` avec `inset-inline-start` négatif.
- Pour votre culture : les *container style queries* (`@container style(--x: y)`) existent mais **ne sont pas Baseline** ; ne les utilisez pas dans le projet.

Critères de réussite

Auto-évaluez-vous avec cette liste ; l'enseignant la vérifie en une minute, page ouverte :

- ☐ `style.css` ne contient **aucune** `@media` de largeur (`min-width` , `max-width` , `width`) — recherche Ctrl+F « @media ».
- ☐ À **1280 px**, les 3 contextes montrent **3 mises en page différentes** (compacte / verticale / horizontale).
- ☐ À **375 px**, aucune barre de défilement horizontale, et les cartes de la barre latérale ont changé de mise en page.
- ☐ Le titre des cartes utilise **`cqi`** dans un `clamp()` ; le héros utilise `dvh`.
- ☐ Avec `dir="rtl"` sur `<html>`, le liseré, l'heure et le visuel sont **du bon côté** (aucune propriété `left` / `right` / `margin-left` … restante).
- ☐ Navigation au **clavier** : focus visible sur chaque lien ; boutons d'au moins **44 px** de haut.
- ☐ Texte secondaire et « places restantes » à **≥ 4,5:1** ; avec `prefers-reduced-motion: reduce`, plus rien ne bouge.

Exercice d'approfondissement (facultatif) : réussi si la frise apparaît dans la barre latérale à 1280 px, disparaît à 375 px et reste correcte en RTL, sans classe ajoutée dans le HTML.

Questions de réflexion (à l'oral, environ 2 min par groupe)

1. Dans quel cas une **media query** reste-t-elle le bon outil, et dans quel cas faut-il une **container query** ? Donnez un exemple de chaque tiré de la page.
2. Pourquoi a-t-on dû déclarer le conteneur sur `.event-card` et mettre en page `.event-card__inner`, plutôt que de tout faire sur un seul élément ? Que se passerait-il avec `container-type: size` ?
3. Votre site doit être traduit en arabe demain matin. Qu'est-ce que les propriétés logiques vous font gagner, et qu'est-ce qui resterait malgré tout à vérifier (icônes, flèches, etc.) ?